Katholieke
Universiteit
Leuven

**Department of
Computer Science**

A Reverse-Hoops-Playing Robot Arm

Robotics (B-KUL-H02A4A)

Enmin Lin (r0927480)

Academic year 2025–2026

Reverse-Hoops-Playing Robot Arm

Setup and Experiment Design

The project asks the robot to move a hoop so that a flying ball passes through it. I treat the problem as one estimation and constrained planning pipeline. The system receives noisy Cartesian ball measurements, predicts the future ball path, selects candidate catch points, solves a UR10 trajectory optimization problem, and checks whether the ball center crosses the hoop plane inside the physical opening.

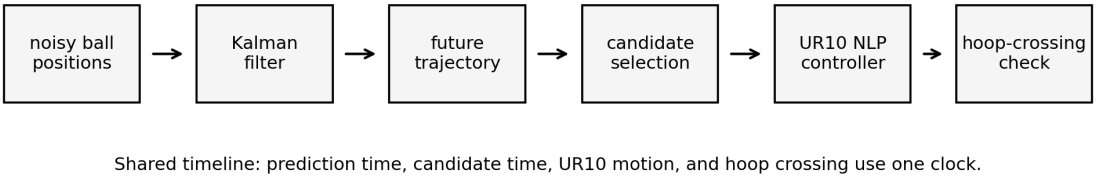

Figure 1: Pipeline used in all experiments. The candidate time, planned UR10 motion, and hoop-crossing check share the same clock.

Table 1: Project constants and modelling choices used in the experiments.

Item	Value	Role in the report
Ball radius	0.12 m	Defines the center clearance through the hoop
Hoop radius	0.15 m	Gives 0.03 m radial tolerance
Measurement noise	1 mm Gaussian position noise	Drives the estimator design
Robot base	Fixed	Removes mobile-base planning from the task
Acceleration limit	2 rad/s^2	Hard NLP bound and benchmark diagnostic
Collision evidence	Table clearance and self-sphere proxy	Conservative check, not exact mesh proof

I use the four project tasks as separate experiments. Task 1 tests prediction error. Task 2 creates a simple geometric baseline. Task 3 tests whether the UR10 can execute a constrained hoop motion. Task 4 compares candidate-selection policies on the same predicted trajectories and the same success definition.

Task 1: Predicting the Ball

The ball state uses position and velocity,

$$x = \begin{bmatrix} p_x & p_y & p_z & v_x & v_y & v_z \end{bmatrix}^\top .$$

The process model integrates constant horizontal velocity and known gravity in z. The measurement model observes position only. Under these assumptions, a linear Kalman filter gives the exact Gaussian Bayesian update for a linear system with Gaussian process and measurement noise. This is the main reason for using it. An EKF would reduce to the same linear update, and a particle filter would add sampling noise to a six-dimensional problem that already has an analytic covariance update.

The validation reports measurement error, filtered error, future-prediction error, and covariance validity. Over 50 seeds at the project noise level, the raw measurement RMSE was 1.74 mm, the filtered RMSE was 0.90 mm, and the 0.5 s future-prediction RMSE was 3.51 mm. The 95th percentile future-prediction RMSE was 6.35 mm, far below the 30 mm hoop clearance.

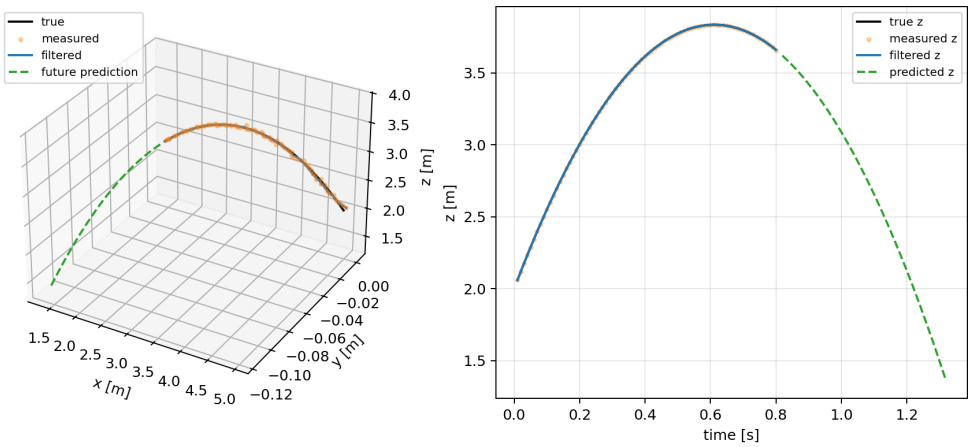

Figure 2: Kalman-filter prediction for one seed. The future prediction continues the gravity-driven parabola from the filtered state.

I also tested whether this conclusion depends on the exact 1 mm noise value. Figure 3 sweeps measurement noise from 1 to 10 mm. The mean 0.5 s prediction RMSE rises from 3.67 to 10.34 mm, and the maximum observed RMSE remains 20.01 mm. The predictor still stays below the hoop clearance in this sweep. This does not prove arbitrary robustness, but it shows that the filter has useful margin around the stated simulator noise.

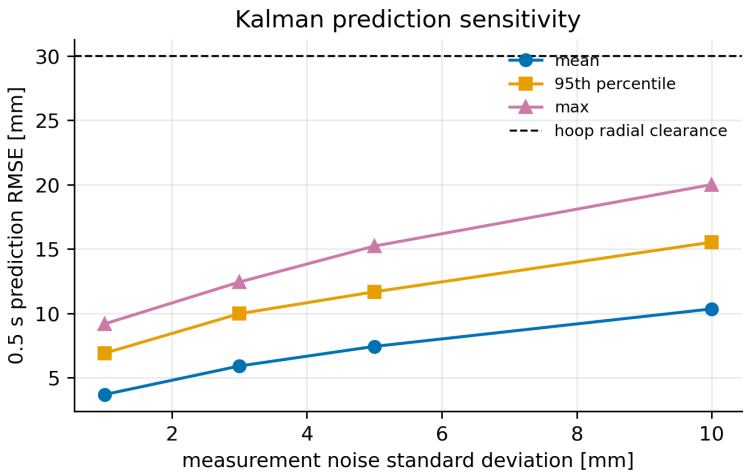

Figure 3: Task 1 noise sensitivity over 50 seeds per noise level. The dashed line marks the 30 mm radial clearance left by the ball and hoop radii.

Task 2: Simple Interception Point

Task 2 asks for a simple interception strategy, so I deliberately use a geometric rule instead of robot optimization. The selector samples the predicted trajectory and rejects points that occur too soon, sit below $z = 0.35$ m, or lie too far from the current hoop center. It then chooses the earliest remaining point. For the validation seed, this rule selected $t = 1.36$ s and position $[1.224, -0.119, 1.074]$ m.

This baseline has a clear purpose: it answers where the ball could be caught in Cartesian space. It does not answer whether the UR10 can move the hoop there quickly enough. The later benchmark keeps this baseline honest. Across 50 seeds, the simple selector attempts an early catch at 1.295 s on average, but none of those attempts pass the full NLP and hoop-crossing check. The attempted time is useful as a lower bound, not as a successful interception time.

Task 3: Optimization-Based UR10 Control

The controller solves a finite-horizon nonlinear program over joint position, velocity, and acceleration,

$$\{q_k, \dot{q}_k\}_{k=0}^{N}, \quad \{\ddot{q}_k\}_{k=0}^{N-1}.$$

It uses the discrete second-order dynamics

$$q_{k+1} = q_k + \dot{q}_k \Delta t + \tfrac{1}{2}\ddot{q}_k \Delta t^2, \qquad \dot{q}_{k+1} = \dot{q}_k + \ddot{q}_k \Delta t.$$

The optimization problem is

$$
\begin{aligned}
\min_{q,\dot{q},\ddot{q}} \; & w_p \|p_{\mathrm{tcp}}(q_N) - p^\star\|_2^2 + \sum_{k=0}^{N-1} w_u \|\ddot{q}_k\|_2^2 + \sum_{k=1}^{N} w_v \|\dot{q}_k\|_2^2 + w_o (1 - R_{zz}(q_N))^2 \\
\text{s.t. } & q_0 = q_{\mathrm{init}}, \quad \dot{q}_0 = \dot{q}_{\mathrm{init}}, \\
& q_{\min} \le q_k \le q_{\max}, \quad -\dot{q}_{\max} \le \dot{q}_k \le \dot{q}_{\max}, \quad -\ddot{q}_{\max} \le \ddot{q}_k \le \ddot{q}_{\max}, \\
& z_{\mathrm{tcp}}(q_k) \ge h_{\mathrm{table}} + m_{\mathrm{tcp}}.
\end{aligned}
$$

The validation uses $N = 28$, $\Delta t = 0.05\,\mathrm{s}$, $w_p = 1500$, $w_u = 2 \times 10^{-2}$, $w_v = 10^{-3}$, and $w_o = 10$. Pinocchio supplies the URDF-based forward kinematics, including the TCP position and rotation. CasADi builds the symbolic nonlinear program, and IPOPT solves it.

For the Task 2 target, the NLP reduced the hoop-target distance from 0.731 m to 0.020 m. It solved in 0.075 s and 14 IPOPT iterations on the test machine. The maximum joint acceleration reached $1.9996\,\mathrm{rad/s^2}$, and the maximum velocity ratio was 0.640. The planned trajectory respected all position, velocity, and acceleration bounds.

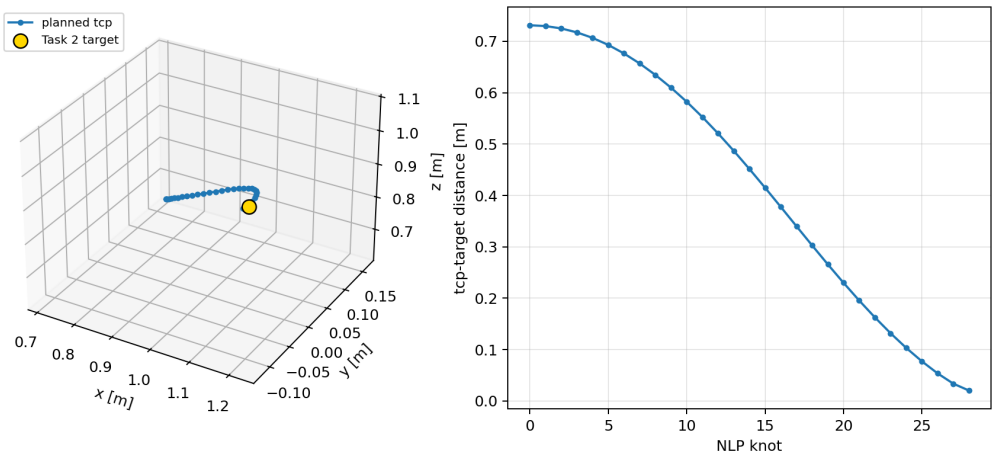

Figure 4: Task 3 finite-horizon NLP result. The planned hoop center moves toward the selected Task 2 target and reaches 20.1 mm terminal TCP error.

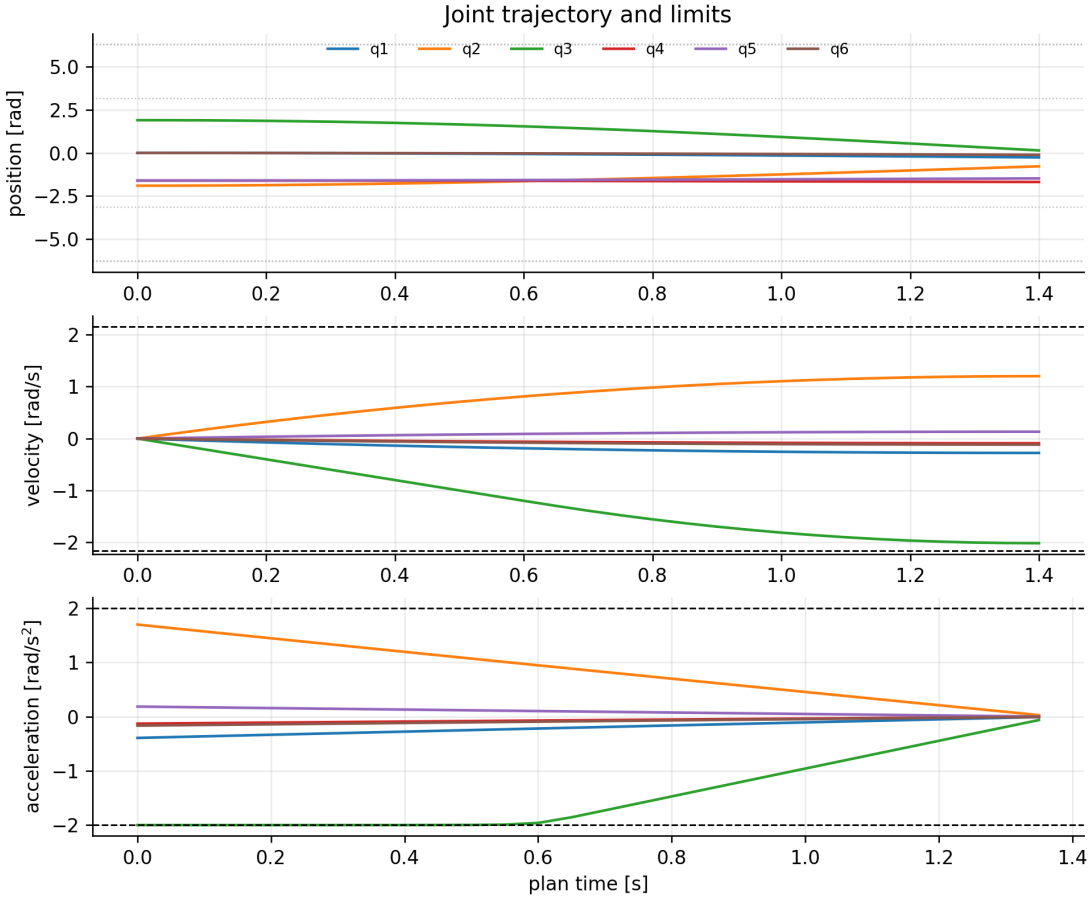

Figure 5: Joint position, velocity, and acceleration profiles for the same Task 3 plan. Dashed lines mark the velocity and acceleration limits. Dotted horizontal lines mark joint-position limits.

The safety checks support the same conclusion. The minimum TCP-table clearance was 0.622 m, the minimum frame-table clearance was 0.115 m, and the minimum self-sphere clearance was 0.163 m. These checks do not replace exact mesh collision detection. They give conservative evidence that the submitted motion does not use a visibly unsafe configuration.

Task 4: Smarter Interception

Task 4 asks for a smarter strategy that improves success rate and interception time. I do not compare the final strategy against a successful Task 2 time, because the simple strategy has no successful catches under the full criterion. Instead, I report the Task 2 attempted time and then evaluate feasible strategies that can actually catch the ball.

The smarter strategy changes the decision rule, not the simulator. It forms a shared candidate pool from the predicted trajectory, runs the Task 3 NLP for each shortlisted candidate, and accepts only candidates that satisfy three conditions: the NLP succeeds, the ball crosses the hoop plane, and the radial crossing error stays within 0.03 m. The time-first feasible policy then selects the earliest candidate that passes these tests. I also report a balanced-cost policy as an ablation. It uses the same candidate evaluations but trades extra catch time for lower radial error and lower control usage.

I added a small pose-aware term for Task 4 candidate evaluation. The term encourages the hoop normal to align with the local ball-flight direction through $w_n(1 - (n_{\text{hoop}}^{\top}\hat{v}_{\text{ball}})^2)$, with $w_n = 5$. This term follows the physical success metric: a ball passes through a hoop opening, not just through a point in space. A larger weight reduced success in tuning, so I kept it as a weak geometric preference rather than a dominant objective.

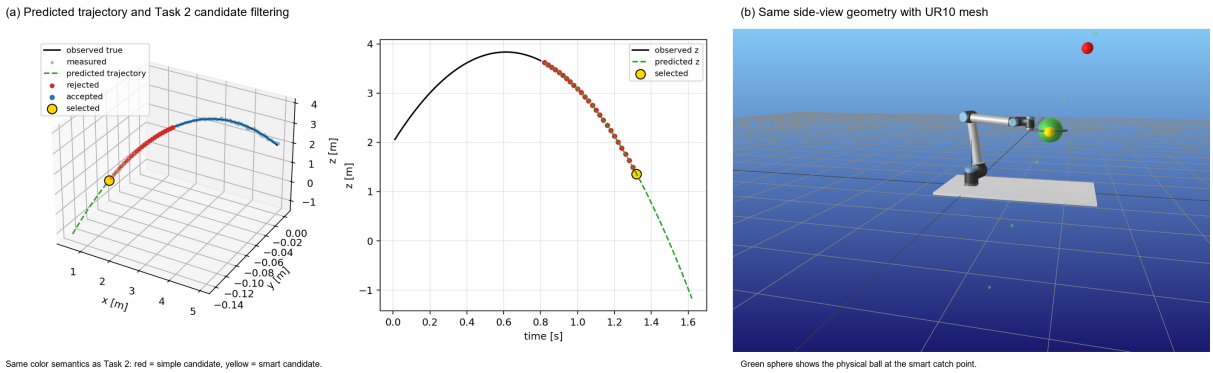

Figure 6: Task 2 and Task 4 geometry in a shared side view. Red marks the simple geometric candidate, yellow marks the smarter selected candidate, and green marks the ball near the catch.

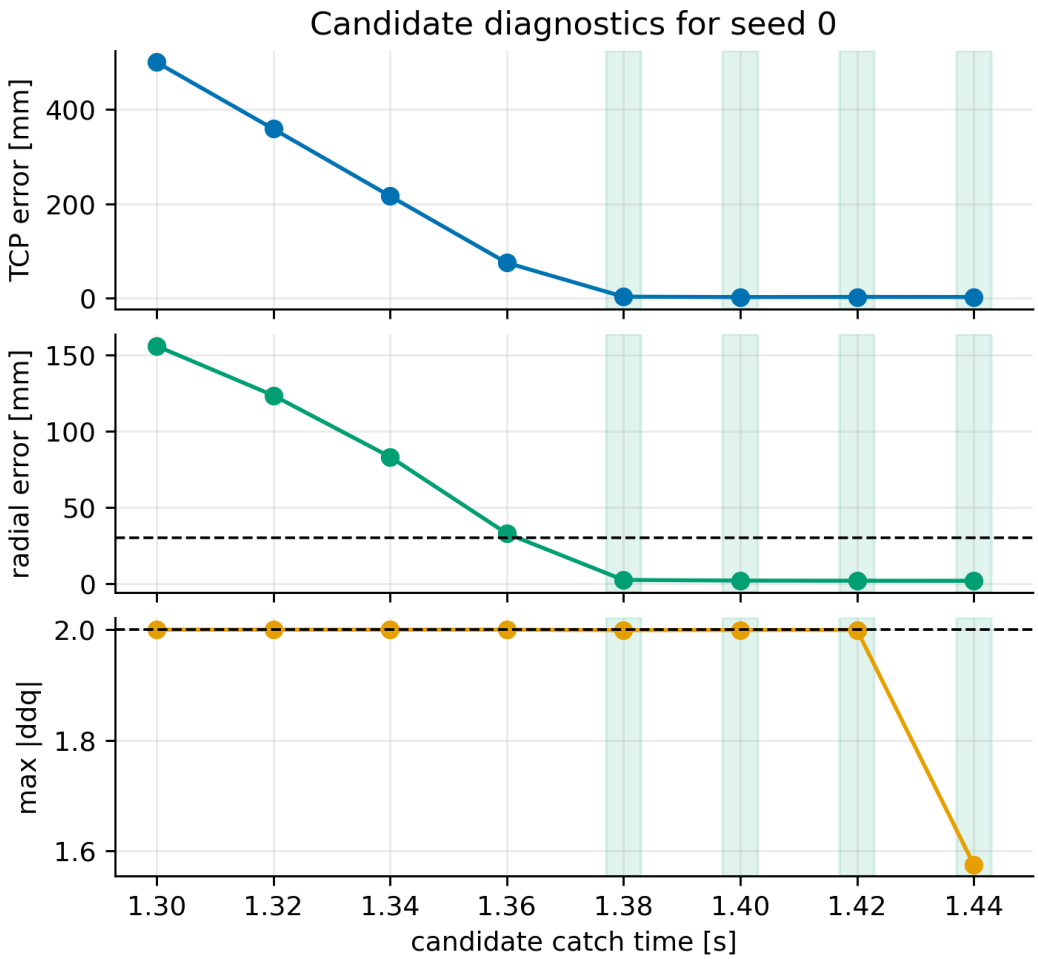

Figure 7: Candidate diagnostics for seed 0. Green vertical bands indicate successful candidates. The early points look attractive in time, but they miss the hoop plane or require saturated motion.

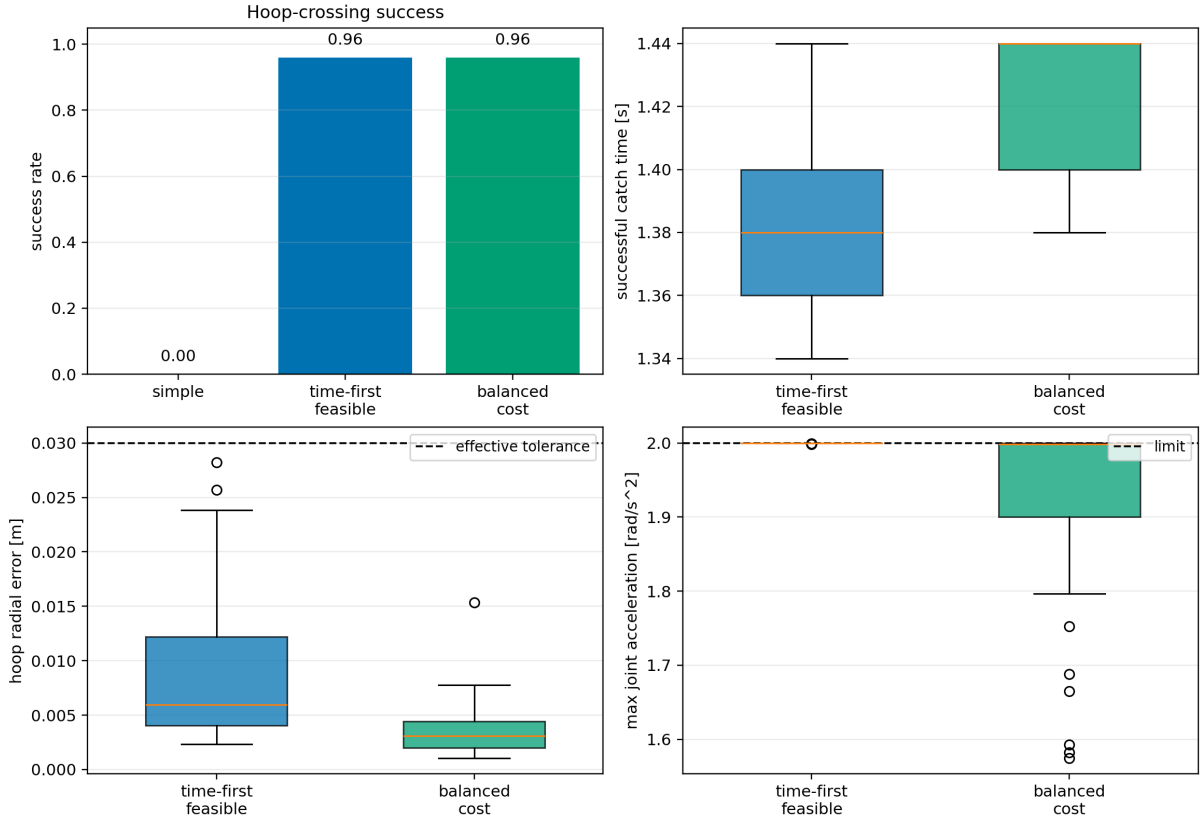

Figure 8: Fifty-seed strategy benchmark. The time-first feasible policy provides the report's main Task 4 result. The balanced-cost policy shows the accuracy-effort tradeoff.

Table 2: Final 50-seed benchmark. Time, errors, effort, solve time, and iterations use successful cases except for the simple attempted time.

Strategy	Success	Time [s]	TCP [mm]	Radial [mm]	Max $\ddot{q}$	IPOPT
Simple geometric	0%	1.295*	–	–	–	–
Time-first feasible	96%	1.383	16.5	9.21	2.00	14.0 it.
Balanced cost	96%	1.421	2.02	3.62	1.93	13.7 it.

*The simple time is an attempted catch time, not a successful interception time.

The main Task 4 result is the time-first feasible policy. It raises the full-system success rate from 0% for the simple baseline to 96%, while preserving the earliest successful catch among evaluated candidates. The balanced-cost policy keeps the same success rate and improves radial error from 9.21 to 3.62 mm, but it catches later. This experiment directly addresses the earlier ambiguity: the submitted "smarter" answer for time is the feasibility-aware time-first policy, and the balanced policy demonstrates the tradeoff rather than replacing the time objective.

Failure Analysis and Limits

Two of the 50 seeds fail for the time-first feasible policy. Figure 9 shows why. Their candidate errors remain outside the 30 mm crossing tolerance until the latest tested candidates, and the candidate pool ends before a robust feasible crossing appears. This is a planning-window and candidate-pool limitation, not a Kalman-filter failure.

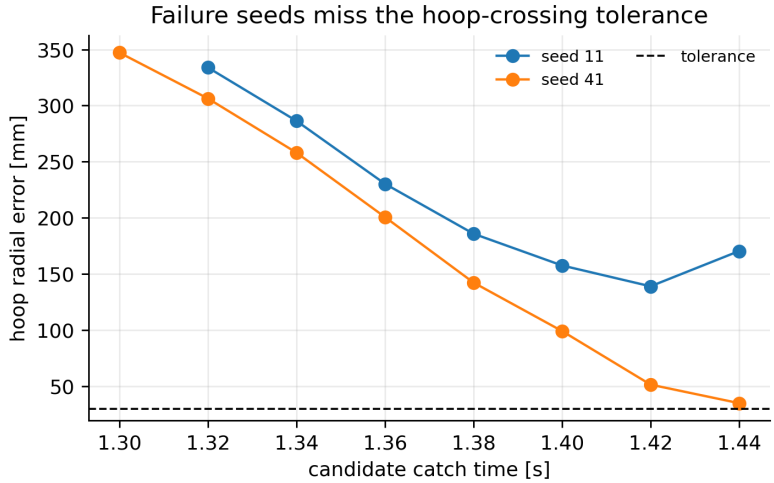

Figure 9: Failure-case diagnosis for seeds 11 and 41. The radial crossing error remains above tolerance for the evaluated feasible window.

The simulator includes a ground bounce model. The Kalman model used here assumes pre-bounce ballistic flight. If a selected trajectory crosses a bounce, the estimator would need a switching model or an interacting multiple-model filter. I keep the submitted experiments in the pre-bounce catching regime. The collision evidence also remains approximate, because the implementation checks table clearance and sphere-based self-clearance rather than exact mesh collision.

Conclusion

The final system gives a complete offline solution for the reverse-hoops task. The Kalman filter predicts the ball with millimetre-level error under the stated noise. The Task 2 rule provides a transparent geometric baseline. The Task 3 NLP produces a bounded UR10 motion with explicit dynamics, joint limits, safety checks, and solver diagnostics. The Task 4 time-first feasible policy gives the strongest final result: 96% full hoop-crossing success across 50 seeds at 1.383 s mean successful catch time. The balanced-cost ablation shows how much accuracy one can gain by spending roughly 0.038 s of extra catch time.

References

1. Robotics course project description, *A Reverse-Hoops-Playing Robot Arm*, KU Leuven, 2025–2026.
2. KU Leuven Robotics lecture C6, *Probabilistic Robotics*: Bayes filters, Kalman filters, and tracking process models.
3. KU Leuven Robotics lecture C3, *Task-Space Control*: resolved motion control, resolved acceleration control, damped pseudo-inverse, and stack of tasks.
4. KU Leuven Robotics lecture C4, *Constraint-Based Robot Programming*: hard and soft constraints, second-order constraints, and instantaneous optimization.
5. KU Leuven Robotics lecture C5, *Motion Planning*: configuration space, free space, signed distance, sampling, and optimal control.
6. R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering*, 82(1), 35–45, 1960. DOI: 10.1115/1.3662552.
7. A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical Programming*, 106, 25–57, 2006. DOI: 10.1007/s10107-004-0559-y.
8. J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi: a software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, 11, 1–36, 2019. DOI: 10.1007/s12532-018-0139-4.
9. J. Carpentier et al., "The Pinocchio C++ library: a fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives," IEEE/SICE International Symposium on System Integration, 2019.
10. R. Featherstone, *Rigid Body Dynamics Algorithms*. Springer, 2008. DOI: 10.1007/978-1-4899-7560-7.